Day01_basic1

- 自我介绍
  - 可以自己想一下，工作中需要什么技能。在这几个月，多锻炼。
    1.基础知识。上课讲+晚上时候梳理
    2.实际代码能力。晚自习的时候会敲代码。 leetcode。写项目的时候
    3.找出问题，解决问题的能力。 debug
    4.沟通能力。 和别人沟通，打交道的能力。
    5.学习新东西的能力
    - **以终为始** — 现在不好好敲代码，做项目时候抓瞎。
      做项目抓瞎，找到工作了要么费很大力气过试用期；要么过不了试用期；要么自己害怕过不了
    - 后期分小组做项目+学习
      - 写代码一起写代码。
      - 锻炼表达能力
      - 会定期开组会，有一个分享学习内容，其实也在锻炼沟通表达能力
      - 后期小组的凝聚力是比较强的。 碰到问题有后援。
      - **也是在锻炼沟通协作能力**
    - 快速学习能力
  - **还有一个。要会回答问题，有些东西你懂，
    但是无法用语言组织出来。这时候就比较被动。
    因为面试的时候，你就涉及到这样一个场景。**
    - 链表。
      同学，简单聊聊链表。
      链表，是一个线性表。 是一个离散的线性表， 链表的每个数据可以不装在一起，是靠一个指针指向的下一个数据。 数组是另外一种格式的线性表，它和链表的区别是，它的数据是存在一起的。
      链表的插入，就是找到前驱结点。然后挪动前驱结点的指针，靠挪动指针完成插入。 删除也是类似。 查找，没有随机访问的特点，只能一个一个的访问， 最终访问到第n个。
      数组的插入， 把后面的数据往后挪一位，然后将要插入的数据放进去。删除，也是挪数据，将数据整体往前挪一位。 查找第n个数据，只需要根据公式直接算出地址，然后直接去该地址取数据。
    - 而且还不能是以背诵的方式，如果是以背诵的方式，基本上面试官一听就知道你是背的。
      接下来面试官会更换其他的不是背诵能解决的问题。换句话，会用别的方式来试探你的水平
    - 先大后小，先回答大的，概念性的东西，再回答细节。 不要糅在一起回答。
  - 图的科普
  - 大数据的科普
    - hadoop
      - 核心思想就是： 拆块， 冗余
      - hadoop是一个别人已经实现好的框架（软件）。
    - mapreduce — 用来解决大数据的计算问题。
  - 用户画像系统
    - 用户画像后端开发工程师
    - 人群包。就是用来找给谁发短信的。
      - 用户年龄在 17-35之间
      - 性别： 女
      - 现在，你是卖化妆品。 需要找到精准的人群，发短信。而不是漫无目的的发
  - **关于学习的几个建议**
    - 1.主动学习。多花点时间学习 — 看文档，动手练。
      看一些基础点的书
      刷一下leetcode。
    - 2.培训不是交了钱坐这就完事了。 — 自己一定要多花时间，肯花时间。
    - 3.基础问题
      - 基础好，不错。多刷刷题
      - 基础差，多花点时间将代码多敲敲
    - 4.代码问题
      - 一定要多敲代码、多练习
      - **敲代码！敲代码！敲代码！**
      - 将白天上课的代码敲一下。 — 1周-2周 差不多能开始敲代码。
    - 5.课堂纪律
      - 上课要听讲，认真一点。
      - 不要戴耳机
      - 不要看其他的游戏视频、娱乐视频、与学习无关的
    - 6.学习方式
      - 课前先提前预习一下。 要知道大致明天上什么内容
      - 课上。
      - 课下（晚自习的时候）。
        一定要敲代码。
        第一个环节，建议先把白天上课的内容梳理一下。 看思维导图、精简版笔记、如果还是不会，问；
        第二个环节，敲代码。 首先先敲一下白天上课的代码。 理解一下这个代码在做什么事情。 然后关上代码
        第三个环节，把作业做一下。
        第四个环节， 自己弄个思维导图捋捋。
- 基础知识
  - C历史。几个C语言的版本了解一下
    - C99/C11 语言的版本
    - C易出错，出错提示不友好
    - 所以调试比较关键
  - VS的安装配置。代码模板配置一下。
    几个快捷键大致了解一下
    - ctrl + k , ctrl +d 格式化
      ctrl + x 剪切
      ctrl + d 复制当前行
      ctrl + shift + / 注释
    - 模板晚上要配置一下
    - 启动项目，项目右键---配置启动项目--- 当前选定内容 — **启动项目时，要确认启动的是哪个项目。**
    - 启动的时候，以X86启动。不要以X64启动
    - 怎么创建项目、项目里面的文件夹，启动项目。 看一下文档。
    - 在VS中，一个项目中，目前只能有一个.c 文件。
      如果想写多个.c，后面再处理
    - 5.9可以做一下。
  - 超基础语法
    - 注释
      - 聊到了规范的问题，有一些规范是约定俗成的。
        不这么写也可以，但是后续在我们写完代码，
        会比较麻烦
      - **规范的问题一定要注意。 要养成好的习惯**
      - 注释是给程序员自己留痕的。比如一些比较复杂的逻辑。 对于一些比较关键的点，程序员可以使用注释留下思路。
      - 自己写代码，规范无所谓； 但是一旦进公司，这些都是非常重要的。 可读性至关重要。
    - 标识符和关键字
      - 关键字混个眼熟 — 下次取名字，不要往上面取。
      - 标识符是我们自己给变量取的名字
        - 英文
        - **见名知意**
          - 不光可以解释变量，还可以解释代码。 帮助我们理解代码
          - 一定要注意变量的解释性。
        - 命名风格
          - C语言我们使用下划线进行分割。
          - 驼峰： 就是多个单词像骆驼峰一样，一高一低。简单说就是单词首字母采用大写。以便程序员能看懂多个单词组合起来
          - 大驼峰：首单词的首字母是大写
          - 小驼峰：首单词采用小写
          - 命名风格，就是多个单词组合在一起的时候，怎么和谐一些。
        - **最低要求**
          - 不能是关键字
          - 以 字母 下划线 开头
          - 里面包含的东西 字母 下划线 数字
        - **较高的要求** — 建议大家按照较高要求。 简单说，就是英文+见名知意。再配合命名风格。
    - 变量和常量
      - 常量，我们一般使用常量给变量赋值。
        常见的整数，类型是int。也就是我们写个10。是int类型
        我们写浮点数，是double类型 3.14
        如果我们希望是float。 后面要加f 3.14f
      - 变量
        - 变量有三要素
          - 变量名 — 这个是我们刚讲的标识符问题
          - 变量类型
            - 一个变量类型，就限制了能进行哪些操作，存储的范围。
              比如一个int。 只能存储 -21亿到+21亿
            - int num;
              float num2;
              定义了这样一个类型，说明了什么事情?
          - 变量值 — 给变量初始化值
          - int num = 10;
        - 我们首先研究局部变量
          - 这个也是我们使用的比较多的变量
          - 什么是局部变量 — 就是在{} 内部的，局部位置的变量
          - **作用域**
            - 局部变量只在{}的局部生效，出了括号就没了。所以大家在使用的时候注意
            - 后期的代码中比较复杂的时候，可能大括号内部疯狂嵌套。
            - 如果不注意，有可能预期和代码的实际不符。
          - 默认值
            - 如果不给局部变量赋初始值，这时候它的值是随机的。 可能是0，可能是其他的，可能是脏数据。
            - 但是在VS中，它会做一些填充。这时候看到的是一个固定的值。但这只是vs的行为。
            - int num=10;
            - **一个局部变量，一定建议要初始化再使用。 不要使用没有初始化的局部变量。 否则就是未定义行为**
          - **未定义行为**
            - 在C标准中，没有对这块进行规定。具体的行为就有编译器自行实现，可能是随机值，可能是固定值，可能崩溃，可能编译报错。
              所以，一定要注意未定义行为，写代码的时候，不要写出这种代码。
            - **一定要小心它**
    - 函数
      - 函数的一些类似叫法 — 函数、方法、钩子
      - 函数定义的语法需要注意
        - 返回值类型 函数名(函数形参){
          函数体
          }
        -
          ```
          int add(int a, int b){
              return a + b;
          }
          ```
      - **函数这块的几个概念**
        - 函数头 — 就是只有第一行，没有那个大括号。
          void test1(int num)。这个就是函数头
        - 函数体 — 就是大括号及内部的内容
        - 函数声明 — 函数头加个分号
          int add(int a, int b);
        - 函数定义 — 整个函数就叫做函数定义
      - 函数的几个注意事项
        - 注意1：被调用的函数，放在上面。不然可能会出现错误。 — 如果有些情况无法挪动位置，可以采用先在上面写一个函数声明。
          然后位置不用调整
        - 注意2： 在调用函数的时候，一定要写()。不然不叫函数的调用。
          拓展：这其实是函数指针，就是一个函数的地址
        - 实参，形参
          - 实际参数： 实际调用函数的时候，传入的参数。
          - 形式参数： 放在函数小括号内，充当代表的值
  - 一个C程序是怎么运行起来的
    - 大致来说，分两步：一步是从源代码变成可执行程序
      一步是可执行程序被调用运行起来
    - 所以，我们第一步研究怎么从main.c变成可执行程序。
      第二步我们研究运行起来的内存问题
    - **源代码变成可执行程序分为几步**
      - 预处理 — .c 变.i
      - 编译 — .i 变 .s
      - 汇编 — .s 变 .o
      - 链接 — 把.o文件合起来，变成一个可执行程序，
        在windows里面exe
      - 分步骤在做什么事情
        - **预处理**
          - **预处理就是将 .c文件 转换为 .i文件**
          - 预处理器在执行预处理指令。也会顺手把注释去掉
          - 要知道怎么看 .i文件。
            - 就是做一些配置。
            - 配置完之后，代码无法运行。
          - #include 就是预处理指令 — 先理解为复制粘贴，把别的地方的代码，借过来用一下
          - #define 也是一个
            - 宏定义。 文本替换。之前使用宏定义常量的，用真实值替换它
              #define ONE_FILE_MAX_SIZE 4096
            - 宏函数。类似的
              - 但是需要注意这块的宏函数，在定义的时候，一定要多加点括号
              - 它不管优先级，只管文本替换，所以一定要小心。
                尽量不要有事没事用它。
        - 编译
          - 狭义的编译，就是把 .i文件转换为 .s文件
          - 编译 阶段，会做 词法分析、语法分析、检查工作
            会进行代码优化
            最终，会生成一个汇编的代码
          - 汇编代码，已经对程序员不是很友好了。
        - 汇编
          - 汇编，就是将.s 变成 .o文件
          - 这时候代码，会变成二进制的机器指令，程序员已经无法看懂这块代码了
          - 对于机器是比较友好的。
        - 链接
          - 将.o文件和一些外部的库文件，链接起来，最终生成一个可执行程序
          - printf
    - 运行时的内存问题
      - 运行时，我们认为我们看到的内存，是一块连续的，从0开始的，到最大的。
        这个我们认为是虚拟内存。
      - **大家要知道如果没有这个一个虚拟内存，有什么问题。**
        - 这几个问题都是相同的回答方式
        - 1.虚拟内存有什么好处
        - 2.为什么我们不是直接看到的物理内存
        - **回答方式**
          - 先说问题。 如果我们直接看到的是物理内存，进程间隔离非常困难。 内存碎片难控制，程序员控制内存很困难。
            现在，我们看到的内存是连续的。这时候操作系统可以帮助我们完成进程间隔离。
            这时候，我们也不需要理碎片，操作系统帮助我们处理这个事情。
            程序员只需要知道这块内存是连续的，使用它就可以了
          - **知道一个知识很重要。 但是能条理清楚的回答清楚一个问题，也非常重要。
            或者说尤为重要**
      - 进程在拿到这样一块内存之后，并不是杂乱无章的使用这样一块内存。
        而是采用分门别类的方式来使用。
        - 内存，和生活中的空间一样。 如果随意使用，后续，肯定会杂乱无章。
          系统在拿到一块内存之后，使用的方式是，按块划分，分门别类。
        - 1.知道系统是按区域划分的，内存分为五个区。
          2.大致了解一下名字。
          3.栈和堆。 空间占用大小不一样。 2G
          1M 8M
        - 从上而下，分为五部分。
          这里需要注意一下，是逻辑的关系。有一些编译器可能并没有严格遵循这个顺序。
        - 代码段、数据段、堆、栈、内核区
        - 知道一下大致的作用
          - 代码段就是存放代码的地方。比如函数的机器指令
          - 数据段： 存全局变量的位置
          - 堆： 动态分配的内存，是在堆区里的
          - 栈： 存储函数的调用过程
          - 内核区： 在涉及到一些系统相关的问题时会用到。 比如读取外部的磁盘等
        - 堆和栈的大小需要知道一下。 堆占的比较大，栈区占用小部分
          在有一些程序中，分得2G内存。 栈区可能只有1M（8M）,堆区得到的大小会非常大，可能能到1.6G，1.7G。
        - 现在先掌握上面的，会随着课程不断地学习，深入理解这五块内存区域。
    - 内存地址问题
      - 内存地址：就是虚拟内存上某一个字节的唯一性标识
      - 变量地址： 一个变量也是有地址的，它存储的地址，是变量的首字节地址。
        比如有一个int类型，一般占用四个字节，存储的时候是首字节地址
      - int *存储的，就是int类型的首地址。
        指针，在C里面就是存地址的。
        - & 取一个变量的地址
        - int * 这个是指针
      - 字节顺序
        - 采用的是小端存储法。数据的低位存储在内存的低地址。
        - 大端存储法
        - **因为C程序员是需要经常看内存的，如果不知道它的具体存储方式，
          有可能出现看不懂内存的情况，所以要知道它的存储顺序**
      - 指针大小/地址的大小 — 和应用程序的位数相关。 如果32位应用程序，则指针占用4个字节；
        如果是64位应用程序，则指针占用8个字节
        4字节=32bit（32个0 1） = 8个16进制数
        8字节=64bit （64个0 1）= 16个16进制数
      - 虚拟内存是怎样的内存?